

Audion VS Engine

Portable Windows-инструментарий для **технической видеоподготовки и стилизации** на базе VapourSynth + FFmpeg. **28 пресетов в 4 независимых палитрах**:

Палитра	Пресетов	Что делает
Precision Engine	8	Шумодав (включая SOTA-таргетированный BM3D по теням), дебандинг, контролируемый возврат микрозерна. Технический слой ДО колориста.
Film Looks Engine	7	Эмуляции киноплёнки: 35mm Kodak (250D/500T/50D), 16mm, Super 8, Bleach Bypass, IMAX 70mm, универсальный Cinematic.
Retro Engine	3	Аналоговый характер: VHS / CRT, Camcorder 90s, Polaroid SX-70.
Restoration Engine	10	«Чего нет в Adobe / DaVinci out-of-the-box»: QTGMC, TIVTC, MVTools-MCDeGrain, derainbow, deblock, dehaze + ML (Real-ESRGAN 2x, Soft HD Rebuild 2X, RIFE 60fps, DPIR через vs-mlrt).

Движок — **CLI-оркестратор на Python**, оборачивающий пайплайны `vspipe` | `ffmpeg`.

Лаунчеры — батники с FZF + CMD fallback (зеркальные копии EN и RU).

Первый запуск: доустановить движок

В раздачу не входят VapourSynth, его плагины и модели vs-mlrt. Распространять их мы не вправе — это около семидесяти модулей, у каждого своя лицензия, и почти три гигабайта вместе. Программа ставит их сама, с сайтов авторов, в пару нажатий.

Пока это не сделано, **программа ничего не обрабатает**. Она запускается, но движка за ней нет.

Запустите `builder_main.cmd` (или Start → меню сборки) и выберите по порядку:

№	Пункт меню	Что ставит
10	<code>VAPOURSYNTH</code>	сам движок — обязательно

№	Пункт меню	Что ставит
11	VS PLUGINS	фильтры, которые вызывают пресеты — обязательно
14	VS-MLRT LEAN	нейромодели, обрезанные под вашу видеокарту
15	VS-MLRT FULL	полный набор моделей

Порядок важен: плагинам нужен уже установленный движок.

Пункты 14 и 15 — для карт NVIDIA RTX и необязательны: без них работает всё, кроме нейрофильтров. Берите **VS-MLRT LEAN** — он оставит только то, что ваша карта реально умеет, вместо полного набора под все видеокарты сразу.

FFmpeg и драйвер NVIDIA

Новее не значит лучше. Каждая сборка FFmpeg компилируется под конкретную версию заголовков NVENC, и каждая из них требует своего минимума драйвера. Поставьте самую свежую на драйвер постарше — аппаратное кодирование не ускорится, а перестанет работать.

Сборка FFmpeg	Заголовки NVENC	Минимальный драйвер NVIDIA (Windows)
9.0.1	ffnvcodec n13.1.15.0	610.0
8.0.1	ffnvcodec n13.0.19.0	570.0
7.1.1	ffnvcodec n13.0.19.0	570.0
7.1	ffnvcodec n12.2.72.0	551.76

Обратите внимание на третью строку: 7.1.1 собрана теми же заголовками, что и 8.0.1, и требует те же 570.0 — «откатиться на версию назад» на старом драйвере не даёт ничего. Помогает переход на 7.1 без патча.

Поэтому установщик подбирает сборку по вашему драйверу, а не берёт последнюю. Версии выше прочитаны из README самой сборки, пороги драйверов — из README nv-codec-headers.

Если видеокарты NVIDIA нет, всё это неважно: ставится последняя сборка, а кодирование идёт на процессоре.

Какая сборка идёт в поставке: 8.0.1. Это осознанный выбор, а не забытое обновление.

Большинство машин для монтажа и кодирования сегодня живут на драйверах примерно с 571 по

609; ветка 610 стоит у считанных единиц. И 8.1.x, и 9.x требуют именно её — поставить их значит заявить аппаратное ускорение NVIDIA и не дать его большинству тех, кому оно обещано. В 8.0.1 есть всё, что используют эти программы, и она работает на тех драйверах, которые у людей действительно стоят.

Первоначальная установка (один раз перед Quick start)

Если дерево проекта только что распаковано / клонировано и `system_core/vapoursynth/` / `Tools/ffmpeg/` ещё пусты — запусти `builder_main.cmd` и пройди пункты в таком порядке:

Stage 1 – Оркестратор Python (auto-runs при первом запуске любого *.cmd, но можно прогнать заранее):

[01] BUILD PORTABLE ENV CMD BUILDER (или [03] INSTALL PORTABLE OFFLINE)

Stage 2 – Core VS engine stack (после этого работает non-ML core):

[04] POWERSHELL - portable pwsh 7
* ПРОПУСТИТЬ если `pwsh -v` уже >= 7

[10] VAPOURSYNTH - latest VS stable + свой embedded Python

3.12.x

[11] VS PLUGINS - vsrepo + плагины (требует [10])

[12] FFMPEG - portable ffmpeg

Опциональный MLRT-шаг (только если нужны ML-пресеты):

[14] VS-MLRT LEAN - скачивает MLRT под эту машину
(не входит в core archive)

[15] VS-MLRT FULL - advanced offline / multi-GPU bundle

Stage 3 – Verify (имеет смысл ТОЛЬКО после Stage 2):

[71] VERIFY / DOCTOR - запускает doctor.py end-to-end
(live BM3D CUDA invocation на NVIDIA)

Stage 4 – Опционально, очистка кеша:

[70] CLEAN INSTALL CACHE - освобождает архивы
install\download\ . При повторной
установке само пере-скачается.

Почему [04] раньше времени даёт FAILURE: doctor.py зондирует весь стек (vspipe, плагины, ffmpeg, опциональный CUDA). До Stage 2 этих бинарников ещё нет — ошибки штатные, не баг.

Про портативный PowerShell ([10]): Windows 10/11 несут только Windows PowerShell 5.1, а скрипты `install/*.ps1` используют синтаксис PS 7+ (ternary, null-coalescing). Поэтому **[10] обязателен, кроме случая когда `pwsh -v` уже показывает 7+** на хосте. `.cmd` wrappers автоматически выбирают системный pwsh 7+ поверх портативного, если оба есть.

Быстрый старт

1. Распакуйте проект в любую папку (он portable, установка не нужна).
2. Запустите `launcher_project_ru.cmd` — главный диспетчер: `[P]` Precision / `[F]` Film Looks / `[R]` Retro / `[N]` Restoration / `[A]` Применить профиль к файлу или папке.
3. Закидывайте материал в `input\`, результаты пишутся в `output\`.

Каждый launcher палитры пошагово ведёт: вход → параметры → энкодер → запуск.

Четыре палитры — конкретное содержание

Precision Engine (`cli\launcher_precision_ru.cmd`)

Меню в порядке pipeline-логики — читается сверху вниз как сигнал-флоу:

```
=== Шаг 1 -- Шумодав (сначала чистим) ===
[01] Мягкий шумодав           DFTTest спектральный, лёгкий/средний/сильный
[02] Шумодав теней SOTA *      BM3D (CUDA->CPU) + плавная luma-маска "только тени"
[03] Очистка хромов           DFTTest только на хрома-плоскостях

=== Шаг 2 -- Дебанд (сглаживаем градиенты) ===
[04] Дебанд безопасный        нео_f3kdb лёгкий, без зерна
[05] Дебанд + тонкое зерно     нео_f3kdb + AddGrain микро-восстановление

=== Шаг 3 -- Композиции (полные цепочки) ===
[06] Filmic Rebuild *         шумодав -> дебанд -> зерно по зонам яркости (флагман)
[07] Архивная очистка        нейтральный мастер, без зерна
[08] Пре-грейд подготовка     минимум воздействия для DaVinci Resolve
```

Film Looks Engine (`cli\launcher_film_looks_ru.cmd`)

7 луков в порядке нарастания характера (мягкий → экстремальный):

[01] Cinematic	универсальный мягкий filmic, безопасный default
[02] Film 35mm	Kodak 250D / 500T / 50D варианты
[03] Film 16mm	органичный, более зернистый, поднятые тени
[04] Super 8	самое сильное зерно, выцветшие '70-е
[05] Bleach Bypass	высокий контраст, десатурация ('Se7en')
[06] IMAX 70mm gate weave	large-format -- минимум зерна, лёгкая halation, 1px
[07] Anamorphic Scope (cinemascope)	горизонтальные blue lens flares + teal-cool тени

Все Film Looks используют общую библиотеку (`system_core/vapoursynth/vs-scripts/audion_lib.py`): MTF-софтинг, halation bloom, зональное по luma зерно, гамма-кривая, поднятие чёрного, десатурация.

Retro Engine (`cli\launcher_retro_ru.cmd`)

[01] VHS / CRT	chroma bleed, мягкая оптика, аналоговый шум
[02] Camcorder 90s	мягче VHS, лёгкая переэкспозиция
[03] Polaroid	1970s SX-70: candy-bloom, тёплая кремовость, виньетка

Restoration Engine (`cli\launcher_restoration_ru.cmd`)

«Чего нет в Adobe / DaVinci out-of-the-box». Требуется дополнительные плагины (havsfunc, mvsvfunc, mvtools, tivtc) — ставятся автоматически через `Install-VS-Plugins.cmd`.

=== Восстановление полей ===

[01] QTGMC деинтерлейс NNEDI3 + MVTools; эталонный деинтерлейс для legacy DV/HDV/VHS

[02] TIVTC обратный telecine NTSC 29.97 telecined -> 23.976 progressive (3:2 pulldown removal)

=== MC-шумодав (motion-compensated) ===

[03] MVTools MCDeGrain temporal denoise без потери детализации (как Topaz внутри)

[04] Derainbow / decross NTSC composite chroma cleanup (радуга, dot crawl)

=== Спасение пережатого ===

[05] Deblock H.264 пережатый YouTube / WhatsApp / SD broadcast

[06] Dehaze / локальный контраст clarity без halos и color shift, 16-bit math

=== AI / ML (vs-mlrt; TensorRT на NVIDIA, DirectML везде) ===

[07] Real-ESRGAN 2x upscale ML апскейл 1080p -> ~4K, чёткие лица / fabric texture

[08] Soft HD Rebuild 2X cleanup + ML-реконструкция мягкого / недосэмплированного HD

[09] RIFE 60fps интерполяция ML интерп. кадров 24->60fps, ровнее Optical Flow

[10] DPIR ML шумодав тяжёлый шум / high-ISO без потери текстуры

Core-релиз не должен нести MLRT payload внутри архива. ML-пресеты требуют отдельного install/update шага через `builder_main.cmd` → [14] `VS-MLRT LEAN`: он скачивает backend/model set под текущую машину и очищает только свою подпапку `plugins\vsmlrt`. [15] остаётся advanced full/offline bundle для multi-GPU distribution, но это не default public package. **Правило Full-update:** после обновления VapourSynth или основных VS-плагинов обязательно запустить MLRT installer заново перед smoke ML-пресетов. Backend выбирается автоматически: TensorRT (NVIDIA, optional) → DirectML (любой DX12 GPU) → CPU. OpenVINO/vsoy после распаковки уходит в `system_core\vapoursynth\disabled_plugins\vsmlrt-openvino`, чтобы не ловить Windows `Bad Image 0xc0e90002` при автозагрузке VapourSynth. Cross-vendor стабильно через `audion_lib.vsmlrt_backend_chain()` — на не-NVIDIA не сжигаются 30–120 секунд на безнадёжную TRT engine компиляцию.

Полный per-preset справочник, decision tree, карта install/maintenance скриптов, encoder ladder и pipeline-сценарии — в `GitHub/VapourWiki_RU.md` (English: `GitHub/VapourWiki_EN.md`). Этот документ — канонический детальный справочник; README — пользовательский landing page.

Системные требования

- Windows 10/11 x64
- ~1.5 GB свободного места для core package после распаковки; опциональные MLRT-компоненты требуют дополнительное место после [14]
- Опционально: NVIDIA GPU + driver R525+ для ускоренного bm3dcuda (×10–50 быстрее CPU). Без него BM3D работает на CPU как документированный fallback.
- Системный Python HE нужен — embedded Python в runtime/.
- Системный FFmpeg HE нужен — portable FFmpeg в Tools/ffmpeg/.

Если что-то отсутствует, запустите launcher_project_ru.cmd → [D] Доктор для диагностики стека.

Архитектура (кратко)

В проекте **два независимых embedded Python**:

runtime/python.exe	# Оркестратор (latest Python 3.12.x)
system_core/vapoursynth/python.exe	# VS-host (latest Python 3.12.x) --
запускает vspipe + плагины	

Оркестратор никогда не импортирует VapourSynth. Он зовёт

system_core/vapoursynth/Scripts/vspipe.exe через subprocess и пайпит y4m в Tools/ffmpeg/bin/ffmpeg.exe.

Важно для VS R74+: реальная директория автозагрузки плагинов берётся через

vapoursynth.get_plugin_dir() и в wheel-layout находится внутри Lib\site-packages\vapoursynth\plugins\ . Старая system_core\vapoursynth\vs-plugins\ — legacy; она может быть пустой и не должна использоваться для install/status.

```
launcher_*.cmd → runtime/python.exe system_core/main.py run \
    --palette X --preset Y --input ... --output ...
↓
↓ subprocess: vspipe -c y4m preset.vpy - | ffmpeg -i - ... output
↓
↓ все .vpy пресеты читают параметры из AUDION_VS_* env vars
↓
↓ JSON-отчёт в logs/{ts}__{palette}__{preset}__{stem}.json
```

Плагины (ставятся автоматически через `Install-VS-Plugins.cmd`):

- v1.0 набор: `lsmas`, `ffms2`, `fmtconv`, `neo_f3kdb`, `addgrain` (namespace `grain`), `knlmeanscl` (`knlm`), `bm3dcpu`, `dfttest`
- `bm3dcuda` (NVIDIA acceleration; **по умолчанию с Phase 18.A0**, opt-out через `/NO-CUDA`)
- Restoration набор (Phase 18.B): `havsfunc`, `mvsfunc`, `mvtools`, `tivtc`, `znedi3`

CLI-справочник (продвинутый режим)

```
runtime\python.exe system_core\main.py <команда> [args]
```

Команда	Что делает
<code>info</code>	Резолвленные пути, версии Python, число загруженных plugin-namespace'ов
<code>doctor</code>	Запустить <code>system_core/doctor.py</code> — полная диагностика стека (Python'ы, vspipe, плагины, ffmpeg, опц. live-смук CUDA)
<code>list-presets</code>	Все 28 зарегистрированных пресетов в 4 палитрах с дефолтными параметрами и группировкой по палитре
<code>list-encoders</code>	Все 34 профиля энкодеров (software CRF / NVENC / QuickSync / AMF / ProRes / DNxHR) с однострочным описанием
<code>list-profiles</code>	Встроенные (5) + пользовательские профили из <code>config\profiles*.json</code>

Команда	Что делает
<code>materialize-profiles [--force]</code>	Записать 5 встроенных профилей в <code>config\profiles\</code> как редактируемые JSON (идемпотентно; <code>--force</code> перезаписывает)
<code>probe --input X</code>	ffprobe-сводка файла (codec, разрешение, fps, длительность, аудио-стримы, color metadata)
<code>run --palette P --preset Q --input I --output O [params...]</code>	Полная обработка: <code>vspipe → ffmpeg</code>
<code>apply-profile --name N --input I --output O [--no-audio]</code>	Запуск сохранённого профиля для одного файла
<code>apply-profile-batch --name N --input-dir D --output-dir E [--recursive] [--no-mirror] [--overwrite] [--no-audio]</code>	Профиль для папки; с <code>--recursive</code> обходит подпапки, зеркалит дерево источника (opt-out: <code>--no-mirror</code>), пропускает уже обработанные файлы (opt-out: <code>--overwrite</code>)

Часто используемые флаги `run` (прокидываются в `AUDION_VS_*` env vars per `MEMORY.md §7`):

`--strength {light,medium,strong}`, `--sigma <float>`, `--use-cuda 0|1`, `--grain-back <float>`, `--shadow-threshold <0..1>`, `--transition <0..1>`, `--deband-range <int>`, `--grain-shadow / --grain-mid / --grain-high <float>`, `--high-threshold <0..1>`, `--stock {250D,500T,50D}` (film_35mm), `--intensity <float>`, `--field-order {tff,bff}`, `--qtmcmc-preset {Faster,Fast,Medium,Slow,Slower,Placebo}`, `--output-fps {single,double}`, `--radius <int>`, `--thsad <int>`, `--blksize <int>`, `--quant1 / --quant2 <int>`, `--fps-mul <float>`, `--model <name>`, `--tile <int>`, `--backend {auto,trt,ort_dml,ort_cpu}`, `--encoder <profile>`, `--no-audio`. Полный список — `... main.py run --help`.

Примеры:

```

:: Шумодав теней SOTA, BM3D на CPU (Intel/AMF), с возвратом зерна
runtime\python.exe system_core\main.py run ^
    --palette precision --preset shadow_denoise_sota ^
    --input input\dark_clip.mov --output output\clean.mp4 ^
    --sigma 2.5 --grain-back 0.6 --shadow-threshold 0.20 ^
    --encoder h264_crf14 --no-audio

:: Filmic Rebuild – полная композиция Шага 3
runtime\python.exe system_core\main.py run ^
    --palette precision --preset filmic_rebuild ^
    --input input\source.mp4 --output output\filmic.mp4 ^
    --sigma 2.0 --deband-range 14 --encoder h265_crf21 --no-audio

:: Kodak 35mm 500T (вольфрам-баланс), полная интенсивность
runtime\python.exe system_core\main.py run ^
    --palette film_looks --preset film_35mm ^
    --input input\source.mov --output output\film35_500t.mp4 ^
    --stock 500T --intensity 1.0 --encoder prores_422hq

:: VHS/CRT петро
runtime\python.exe system_core\main.py run ^
    --palette retro --preset vhs_crt ^
    --input input\source.mp4 --output output\vhs.mp4 ^
    --intensity 1.2 --encoder h264_crf14

:: Restoration -- QTGMC деинтерлейс legacy DV / HDV / VHS
runtime\python.exe system_core\main.py run ^
    --palette restoration --preset qtgmc_deinterlace ^
    --input input\interlaced.mov --output output\progressive.mp4 ^
    --field-order tff --qtgmc-preset Medium --output-fps single ^
    --encoder prores_lt --no-audio

:: Restoration -- MVTools-MCDeGrain temporal denoise (без потери детализации)
runtime\python.exe system_core\main.py run ^
    --palette restoration --preset mvtools_mcdegrain ^
    --input input\noisy_iso6400.mp4 --output output\clean_detail.mp4 ^
    --radius 2 --thsad 200 --blksize 16 --encoder h264_crf14 --no-audio

```

Профили энкодеров (21 всего, ladder 14/17/21): software `h264_crf{14,17,21}` / `h265_crf{14,17,21}` (default `h264_crf14`); NVENC hardware `h264_nvenc_q{14,17,21}` /

`h265_nvenc_q{14,17,21}`; ProRes `prores_lt` (Audion default) / `prores_lt_mxf` / `prores_422` / `prores_422hq` / `prores_422hq_mxf`; DNxHR `dnxhr_lb/sq/hq/hqx`.

Профили (сохранённые комбинации)

Профили позволяют сохранить связку палитра + пресет + параметры + энкодер под именем и применять одной командой. **Встроенные профили в составе v1.0:**

Имя	Что делает
<code>shadow_clean_quick</code>	Быстрая очистка теней на тёмном цифровом материале
<code>filmic_warm_35mm</code>	Kodak 250D 35mm filmic look на интенсивности 1.0
<code>archival_master</code>	Нейтральный архивный мастер, ProRes LT (или HQ для keying)
<code>resolve_handoff</code>	Минимум воздействия, ProRes LT (MXF wrapper доступен) для DaVinci / Avid
<code>vhs_dreamy</code>	VHS/CRT ретро на интенсивности 1.2

Использование:

```
runtime\python.exe system_core\main.py list-profiles
runtime\python.exe system_core\main.py apply-profile --name filmic_warm_35mm ^
--input input\source.mov --output output\filmic.mp4
```

Пользовательские профили — JSON-файлы в `config\profiles\*.json` (перекрывают встроенные по имени). Формат:

```
{
  "palette": "precision",
  "preset": "shadow_denoise_sota",
  "params": { "sigma": 2.5, "grain_back": 0.6, "shadow_threshold": 0.20 },
  "encoder": "h264_crf14",
  "description": "Очистка ночного материала под мой свет"
}
```

Портативность

Проект **полностью портативен**. Скопируйте всю папку на любой Windows-диск/машину и работаете. Никаких installer'ов, системного Python, изменений PATH/реестра — все компоненты (оба embedded Python'a, VapourSynth host, ffmpeg, fzf, portable PowerShell 7) живут внутри `system_core/`. После переезда — **один шаг**: `install\Repair-PipShims.cmd` (чинит `pip-shebang` в `Scripts/*.exe`, см. раздел *Диагностика и обслуживание* ниже). Единственная out-of-tree зависимость — видеодрайвер NVIDIA, если нужен CUDA.

Настройка CUDA (только для NVIDIA)

Чтобы включить `bm3dcuda`, нужен один из двух вариантов:

- **NVIDIA Studio Driver R525+** (Game Ready тоже работает; Studio предпочтительнее ради стабильности) — драйвер сам ставит `cuda64_12.dll` и `cufft64_*.dll` в `C:\Windows\System32\`, это всё что нужно `bm3dcuda` в рантайме. **ИЛИ**
- **Драйвер + CUDA Toolkit 12.x или 13.x Network installer, компонент "Runtime libraries" (~300 МБ)** — нужно в редких случаях, когда driver-bundled рантайм не подходит сборке плагина (например на новейших Blackwell sm_120 мы использовали Toolkit 13.2.3 для чистой JIT-компиляции PTX).

Полный CUDA SDK (~3 ГБ), cuDNN и TensorRT **не нужны**.

С каким GPU NVIDIA работает

`bm3dcuda` собран против CUDA 12.x; формальный минимум — Compute Capability ≥ 5.0 (Maxwell, 2014). На практике:

Поколение	Compute	Карты	Замечание
Maxwell	5.0–5.2	GTX 750/750 Ti, GTX 9xx	Работает, но мало VRAM и почти нет выигрыша vs CPU
Pascal	6.0–6.1	GTX 10-series	Практический минимум. 4–8 ГБ VRAM, заметный gain на 1080p/4K
Turing	7.5	GTX 1650/1660, RTX 20-series	Хорошо. Field-tested на GTX 1650 Super
Ampere	8.6	RTX 30-series	Сильный gain на 4K

Поколение	Compute	Карты	Замечание
Ada Lovelace	8.9	RTX 4070/4080/4090	Sweet spot для регулярной 4K-работы
Blackwell	12.0	RTX 5070/5080/5090	Verified live: RTX 5070 + BM3DCUDA R2.15 + CUDA Toolkit 13.2.3

Для 4K BM3D комфортно ≥ 4 ГБ VRAM.

Где CUDA раскрывается сильнее (и где не помогает)

CUDA-gain масштабируется с **разрешением**, **длиной клипа** и **долей BM3D в пресете**:

- **Разрешение.** BM3D = $O(\text{пикселей})$; GPU launch overhead — константа. На 4K относительный gain в 1.5–2× больше чем на 1080p.
- **Длина клипа.** До ~5 секунд JIT/setup cost не амортизируется; от ~10 секунд gain стабилизируется.
- **Вес пресета.** `shadow_denoise_sota` и `filmic_rebuild` — почти весь wall-time это BM3D, поэтому максимальный gain. `mild_denoise`, `chroma_cleanup`, `deband_safe`, `pregrade_prep` используют DFTTest / neo_f3kdb (CPU-only) — CUDA там не помогает.
- **Контейнер/кодек источника.** **Не** влияет в pure pipeline (H.264, HEVC, ProRes дают одинаковую тенденцию при равном разрешении). Разница появляется только когда libx264 CRF18 в той же петле — он насыщает CPU и сглаживает видимую разницу.
- **Поколение GPU.** Каждый шаг Pascal → Turing → Ampere → Ada → Blackwell примерно удваивает абсолютную скорость BM3D; относительный CPU-vs-CUDA gain на одной машине больше зависит от разрешения/длины, чем от поколения GPU.

Reference на RTX 5070 + Ryzen 9 5900X (pure denoise, без энкодинга в петле):

- 1080p × 32 с H.264: shadow_denoise_sota –22%, filmic_rebuild –15%
- DCI 4K × 25 с ProRes: shadow_denoise_sota **–35%**, filmic_rebuild **–24%**

Установка CUDA (шаги)

`bm3dcuda` — опциональный ускоренный BM3D-плагин (×10–50 быстрее CPU на подходящем материале).

1. Поставьте **NVIDIA Studio Driver R525+** (новее лучше). Драйвер сам несёт `cuda64_12.dll` и `cufft64_*.dll` — это всё что нужно `bm3dcuda` в рантайме. **Полный CUDA Toolkit не требуется.**
2. Запустите `install\Install-VS-Plugins.cmd` — поставит все плагины включая `bm3dcuda` (Phase 18.A0 default). Флаг `/NO-CUDA` пропустит CUDA-плагин на машинах без NVIDIA.
3. Запустите `runtime\python.exe system_core\doctor.py` — он реально дёрнет `core.bm3dcuda.BM3D(...)` на синтетическом кадре. Если вернёт `[WARN] bm3dcuda runtime - plugin loaded but CUDA call failed`, поставьте **CUDA Toolkit 12.x Network Runtime** (~300 MB, только компонент "Runtime libraries", а не полный 3 GB SDK).
4. Используйте CUDA-путь в BM3D-пресетах:

```
runtime\python.exe system_core\main.py run ^
--palette precision --preset shadow_denoise_sota ^
--input ... --output ... --use-cuda 1
```

5. (Опц.) Проверьте реальный CUDA-выигрыш на своём железе — перетащите видео на `install\Bench-CUDA.cmd`. См. раздел *Диагностика и обслуживание* ниже.

Диагностика и обслуживание

Три вспомогательных скрипта живут в `install/` рядом с инсталляторами. **Запускать можно когда угодно** — пресеты и output они не трогают.

`install\Repair-PipShims.cmd` — починка `Scripts\*.exe` после переезда

Симптом. После перемещения папки проекта на другой диск/путь (drag-n-drop, распаковка релизного zip, новая машина) `vspipe.exe` и остальные pip-launcher'ы в `system_core\vapoursynth\Scripts\` тихо завершаются с exit 1 без вывода. `doctor` показывает `[FAIL] vspipe runs`, при этом `python.exe -c "import vapoursynth"` работает.

Причина. При установке pip вшивает абсолютный shebang (`#!/<полный путь к python.exe>`) внутри каждого `Scripts\*.exe`. После переезда этот путь больше не

существует.

Что делать. Запустить `install\Repair-PipShims.cmd`. Скрипт перезаписывает shebang во всех `Scripts\*.exe`, направляя его на текущий `system_core\vapoursynth\python.exe`. Stub launcher'a и trailing-zip остаются нетронутыми. Флаг `/WHATIF` (или `/N`) — dry-run без записи.

Уже встроен в `Install-Portable-VapourSynth.ps1` как post-step, поэтому свежая установка не требует ручного запуска — только починка после переезда.

`install\Bench-CUDA.cmd` — pure-pipeline бенч CPU vs CUDA

Что бенчит. Два BM3D-тяжёлых пресета — `shadow_denoise_sota` и `filmic_rebuild` — каждый прогоняется дважды (CPU и CUDA) на видео, которое вы скормили. Pipeline: `vspipe → ffmpeg -f null` (без энкодинга в петле). Это изолирует VapourSynth-шумодав от libx264 на CPU, который иначе на быстром многоядернике съест всё wall-time и спрячет CUDA-выигрыш.

Как использовать. Перетащить видео на `Bench-CUDA.cmd` или передать аргументом: `Bench-CUDA.cmd "C:\clips\test.mov"`. Опционально вторым аргументом sigma (по умолчанию `2.5`).

Вывод. Четыре строки с таймингом и итоговая таблица CPU/CUDA в секундах с `Δ %`. На диск ничего не пишется — это чистое измерение. Откройте Task Manager → Производительность → GPU (или `nvidia-smi -l 1` в другом терминале), чтобы увидеть как CUDA-проход нагружает карту.

Как читать. На референсном клипе DCI 4K ProRes 25 секунд (RTX 5070 / Ryzen 9 5900X): `shadow_denoise_sota` —35% с CUDA, `filmic_rebuild` —24%. На клипах короче ~5 секунд CUDA setup-cost не амортизируется и результат может быть шумом (и даже немного медленнее CPU). Длина, разрешение и доля BM3D в пресете — всё имеет значение.

Про утилизацию GPU. Task Manager часто показывает всего 5–10% GPU даже на работающем CUDA-пути. BM3D bandwidth-bound и работает всплесками; вокруг него `fmtc` bit-depth конвертации и frame-prop тэги выполняются на CPU. Авторитетный сигнал что CUDA реально живой — wall-time delta и строка `doctor.py`: `[OK] bm3dcuda runtime - live BM3D CUDA invocation succeeded`.

`runtime\python.exe system_core\doctor.py` — здоровье стека

Гонится всё время после установочных шагов — проверяет orchestrator + VS-host Python'ы, находит vspipe / vsrepo / fzf / portable pwsh, перечисляет все загруженные плагины (с опциональным live-смоком `bm3dcuda`), подтверждает ffmpeg / ffprobe, репортит NVIDIA / CUDA.

Возвращает non-zero если что-то критическое отсутствует — пригоден для CI / pre-release проверок.

Сборка с нуля

Если стартуете с минимального scripts-only релиза:

1. `builder_main.cmd` → `[01] Build portable env CMD builder` — ставит portable 7-Zip и собирает `runtime/`
2. `builder_main.cmd` → `[04] POWERSHELL` — pwsh в `system_core/powershell/`
3. `builder_main.cmd` → `[10] VAPOURSYNTH` — VS в `system_core/vapoursynth/`
4. `builder_main.cmd` → `[11] VS PLUGINS` — vsrepo поставит все плагины
5. `builder_main.cmd` → `[12] FFMPEG` — применяет exact-stable policy через Gyan; rolling-сборки BtbN доступны только при явном opt-in, установка в `Tools/ffmpeg/`
6. `runtime\python.exe system_core\doctor.py` — должен показать зелёный стек

Полный bootstrap занимает ~5 минут при 100 Мбит (~250 MB загрузок).

Структура проекта

```
Audion VS Engine/
├─ launcher_project.cmd  launcher_project_ru.cmd      Главный диспетчер
├─ cli/
│   └─ launcher_precision.cmd  launcher_precision_ru.cmd  CLI-лаунчеры палитр
│   └─ launcher_precision.cmd  launcher_precision_ru.cmd  Шаг 1/2/3 pipeline (8
пресетов)
│   └─ launcher_film_looks.cmd  launcher_film_looks_ru.cmd  7 плёночных луков
│   └─ launcher_retro.cmd  launcher_retro_ru.cmd          3 ретро лука
│   └─ launcher_restoration.cmd  launcher_restoration_ru.cmd 10 restoration (вкл. 4
ML)
├─ builder_main.cmd  launcher_gui.cmd  launcher_tools.cmd  Сервисные/GUI-лаунчеры
├─ runtime/
оркестратора (latest 3.12.x)
├─ system_core/
│   └─ main.py  doctor.py      CLI + диагностика
│   └─ engine/
logging / presets / profile / selfheal
│   └─ presets/{precision,film_looks,retro,restoration}/  28 .vpy пресетов
│   └─ vapoursynth/
Python 3.12.x + плагины)
│   └─ └─ vs-scripts/audion_lib.py  VS-host (свой latest
has_cuda_gpu / bm3d_auto / vsmlrt_backend_chain)
│   └─ ffmpeg/  общие хелперы (вкл.
(BtbN/Gyan GPL)
│   └─ powershell/  Portable FFmpeg
│   └─ 7zip/7zr.exe  Portable PowerShell 7
│   └─ fzf.exe  Portable 7-Zip CLI
├─ install/  Инсталляторы +
диагностика (.cmd + .ps1):
│   └─
│   {PowerShell,VapourSynth,FFmpeg}, Install-VS-Plugins,
│   └─
│   (опциональный MLRT downloader),
│   └─
│   (storage > bandwidth),
│   └─
│   helper для 7zr/7za),
│   └─
│   shebang после переезда; auto через selfheal),
│   └─
бенч CPU vs CUDA),
```

	Bench-AllPresets (smoke
по всем 28 пресетам, sweep modes),	
	make_release_archive
(релизный zip с исключениями)	
└─ GitHub/	Документация для
публикации:	
	VapourWiki_EN.md /
VapourWiki_RU.md (канонический справочник пресетов и скриптов),	
	README_EN.md /
README_RU.md (этот файл), SECURITY, LICENSE, release notes	
└─ config/	Defaults +
пользовательские профили (`profiles/*.json`)	
└─ input/ output/ logs/ release/	Папки пользователя
└─ CLAUDE.md MEMORY.md	Контекст для AI-агента
(продолжение работы)	

Файлы документации

- **CLAUDE.md** — рабочий контракт для AI-агентов, продолжающих разработку (в корне)
- **MEMORY.md** — полное состояние проекта, архитектура, gotchas, план фаз (в корне)
- **GitHub/README_EN.md** / **README_RU.md** — пользовательский landing page (этот файл)
- **GitHub/VapourWiki_EN.md** / **VapourWiki_RU.md** ★ — **канонический справочник пресетов и скриптов**: decision tree по материалу, per-preset параметры + рекомендуемый энкодер, карта install/maintenance скриптов (когда что запускать), encoder ladder, pipeline-сценарии
- **GitHub/SECURITY.md** — security policy
- **LICENSE** / **GitHub/LICENSE (GPL-3.0-or-later).md** — текст GPLv3 для проектной лицензии (**GPL-3.0-or-later**)
- **GitHub/** прочие файлы — публикационная мета (release notes, project page description, one-liner)

Лицензия

Авторский код Audion, скрипты, лаунчеры, пресеты и документация лицензируются как **GPL-3.0-or-later** (см. **LICENSE**). Сторонние инструменты и библиотеки (VapourSynth, FFmpeg,

плагины, Python, wheels, PowerShell, 7-Zip, fzf и опциональные MLRT-компоненты) остаются под собственными лицензиями; собирай их через `builder_main.cmd` → `[06] Collect release licenses` в `licenses/` и `licenses/THIRD_PARTY_NOTICES.md`.

Статус: v1.0 + Phase 18.A0 / A1 / B / B-ML / C-part / D / E + Soft HD Rebuild ☒ — production-ready на Windows 10/11 x64. Cross-vendor стабильно: CPU/OpenCL fallback проверен на Intel Xe iGPU; исторический CUDA-pipeline проверен на RTX 5070 (Blackwell sm_120) с BM3DCUDA R2.15. Свежий локальный all-presets smoke: **28/28 PASS** (`Frames=1`, `Cuda=off`, `MLBackend=ort_cpu`, 2026-05-16). CUDA/TensorRT sweep для этой точки — handoff на NVIDIA-машину: `Bench-AllPresets.ps1 -Frames 1 -Cuda sweep -MLBackend auto`.

Канонические названия Workbench

Workbench использует единый публичный словарь Audion Image Tools во всех проектах. Кнопки всегда расположены и называются одинаково: **Источник**, **Добавить файл...**, **Назначение**, **Сбросить**, **Удалить**, **Список**.

Сбросить возвращает проектные `input/output` и не удаляет файлы; **Удалить** очищает текущие **Источник** и **Назначение** только после подтверждения. В английском интерфейсе точные названия: **Source**, **Add file...**, **Target**, **Reset**, **Delete**, **List**. Варианты **Цель**, **Очистить**, **Destination** и **Clear** для этих элементов Workbench не используются.